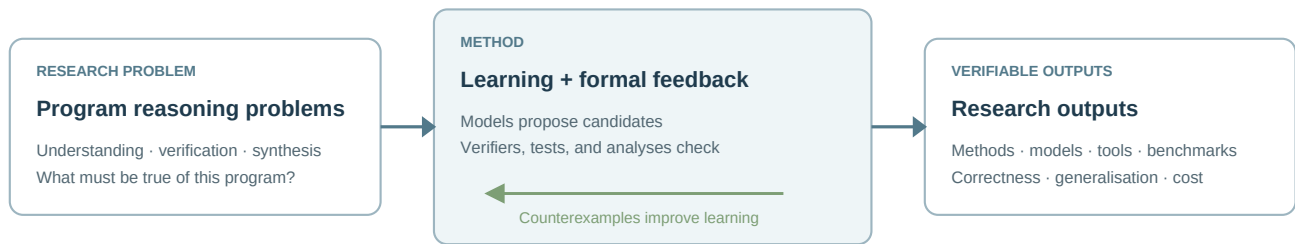


Learning and Formal Reasoning for Program Understanding, Verification, and Synthesis

Jian Wang, PhD · Research statement for Program Reasoning · September 2026



Research problem

The [Program Reasoning project at Imperial Global Singapore](#) asks how AI can reason rigorously about software, code, and formal systems. My central question is: **how can learned models propose useful program abstractions while formal methods determine when those proposals are correct?** I focus on program understanding, verification, and synthesis, using loop invariants and program repair as concrete starting points.

The difficulty is that a code model can produce an invariant or patch that looks plausible but fails on execution or proof obligations. Formal tools give precise decisions, but often need specifications or candidates that are expensive to construct. I want to combine their strengths: models generate candidates; solvers, tests, and static analyses provide feedback; evaluation measures correctness, generalisation, cost, and failure modes.

Proposed research

Verification-guided learning. I would train code models to propose invariants, specifications, summaries, or repairs, then use failed proof obligations and counterexamples as learning signals. Initial experiments would compare supervised fine-tuning, formal-reward reinforcement learning, and counterexample-guided refinement. The generator and checker would remain separate so that a model cannot receive credit simply by exploiting one encoding.

Generalisation across programs and tasks. The first test bed would be loop-invariant synthesis, extending from small single loops to multiple loops, arrays, functions, and C/C++ programs. Evaluation splits would separate program families and include semantics-preserving transformations. I would then test whether learned abstractions transfer to fault localisation and repair: do they constrain a patch, explain a failure, or improve verification of the repaired program?

Practical, auditable outputs. The programme would produce a verification-feedback learning method, a program-reasoning model, a realistic benchmark, and a reproducible evaluation harness. Results would report verifier success, invariant strength, test outcomes, coverage, solver calls, latency, and compute. Negative results would be retained: a method that works for linear arithmetic but fails on arrays should support only the narrower claim.

Evidence from prior work

[Loop-R1](#) is the closest starting point. It builds a semantic pipeline for loop-invariant synthesis and compares supervised learning, reinforcement learning, and their combination. On [Clause2Inv](#), [Qwen3-1.7B](#) improves from 19.39% pass@10 to 61.25% after supervised and reinforcement learning. Performance on [InvBench](#) remains low, and reinforcement learning alone is weak or unstable. These results show both the value of formal feedback and the need for harder generalisation tests; they do not establish a general solution.

My earlier work adds realistic software-engineering evidence. [Defects4C](#) contains 248 real buggy functions and 102 vulnerable functions with executable reproduction tests, and evaluates 24 models on C/C++ repair. Our [execution-trace study](#) found limited gains from trace information in the evaluated fine-tuning and test-time settings, cautioning against assuming that more semantic tokens automatically improve reasoning. [RATCHET](#) combines retrieval, learned fault localisation, and patch generation, providing a baseline for comparing retrieved examples with verifier-produced feedback.

Together, these projects establish experience in code-model training, executable benchmark construction, program-repair systems, and reproducible evaluation. They do not establish new theorem-proving results. At Imperial I would deepen the formal-methods side with programming-languages and verification researchers while contributing the modelling, systems, and benchmark work needed to test ideas at scale.

Fit and contribution

Imperial's project connects AI for code with program analysis, formal verification, automated reasoning, and neuro-symbolic learning. I can contribute at those interfaces: designing and training models, integrating research tools, building benchmarks, and evaluating computational experiments with explicit evidence and costs. My production-engineering background also helps turn research methods into usable and maintainable prototypes.

My initial plan is concrete. First, reproduce and stress-test [Loop-R1](#) with family-level splits and independent checks. Second, develop learning signals from proof failures and counterexamples. Third, transfer the strongest method to C/C++ repair on [Defects4C](#). The intended outputs are publishable methods, open benchmarks and software where appropriate, and a clear record of assumptions and unsuccessful configurations.

My longer-term goal is a system that proposes useful semantic artefacts, learns from formal feedback, and recognises when its evidence is insufficient. It would not replace verification; it would make rigorous program reasoning easier to apply while keeping correctness claims anchored to independently checkable obligations.